RECURSIVIDAD

Guía Práctica

Contenido Sinóptico

Ejercicios para solucionar utilizando recursividad. Comparación con las técnicas iterativas.

---

1. El factorial de un número entero positivo n se define como:

$$n! = \begin{cases} 1 & , \text{si } n = 0 \vee n = 1 \\ n \times (n-1)! & , \text{si } n > 1 \end{cases}$$

   Diseñe un algoritmo recursivo para calcular el factorial de n.

2. El algoritmo de Euclides para hallar el M.C.D. (Máximo Común Divisor) de dos enteros positivos a y b se puede definir recursivamente de la siguiente manera:

$$\text{MCD}(a, b) = \begin{cases} a & , \text{si } b = 0 \\ \text{MCD}(b, a \bmod b) & , \text{si } b \neq 0 \end{cases}$$

   Elabore un algoritmo recursivo que calcule el máximo común divisor de dos enteros positivos a y b.

3. Diseñe un algoritmo recursivo fib(n) para calcular el Fibonacci de un número entero positivo n. Muestre todos los estados por los cuales pasa la pila de registros de activación al ejecutarse en un algoritmo la instrucción f ← fib(4).

$$\text{fib}(n) = \begin{cases} 0 & , \text{si } n = 0 \\ 1 & , \text{si } n = 1 \\ \text{fib}(n-1) + \text{fib}(n-2) & , \text{si } n > 1 \end{cases}$$

4. Elabore un algoritmo recursivo que calcule la suma de dos números naturales a y b.

5. Elabore un algoritmo recursivo que calcule el producto de dos números naturales a y b.

6. Elabore un algoritmo recursivo que dado un número real a y un entero no negativo b calcule a^b.

7. La función de Ackermann se define de manera recursiva para enteros no negativos de la siguiente manera:

$$A(m, n) = \begin{cases} n+1 & , \text{si } m = 0 \\ A(m-1, 1) & , \text{si } m \neq 0 \wedge n = 0 \\ A(m-1, A(m, n-1)) & , \text{si } m \neq 0 \wedge n \neq 0 \end{cases}$$

Escriba una función recursiva que, dados dos enteros no negativos *m* y *n*, calcule el valor de A(*m*, *n*) y a partir de la definición anterior pruebe que A(2, 2) = 7.

8. Dado un número entero positivo *n*, realice un algoritmo recursivo que permita calcular su cantidad de dígitos.

9. Determine qué calcula la siguiente función recursiva:

```
func F(entero n): entero
inicio
   si (n = 0) entonces
         retornar(0)
   sino
         retornar(n + F(n-1))
   fsi
ffunc
```

10. Diseñe un algoritmo recursivo que encuentre la suma de los enteros positivos pares desde *n* hasta 2. Si *n* es impar el algoritmo debe imprimir un mensaje de error.

11. ¿Cuál es el resultado de esta función para distintos valores de *x*?

```
func f(entero x): entero
inicio
   si(x > 100) entonces
         retornar(x – 10)
   sino
         retornar( f( f(x + 11) ) )
   fsi
ffunc
```

12. Elabore un algoritmo recursivo que, dado un entero no negativo *n* calcule su equivalente en binario.

13. Modifique el algoritmo anterior de tal manera que sea capaz de convertir el número en su equivalente en cualquier otra base.

14. Elabore un algoritmo recursivo que transforme un número binario a un número entero positivo.

15. Elabore un algoritmo recursivo que dada una secuencia de *n* enteros calcule la suma de sus elementos.

16. Dada una secuencia de enteros, elabore un algoritmo recursivo que calcule su longitud.

17. Diseñe un algoritmo recursivo que, dado un arreglo de enteros ordenados de menor a mayor y un entero *x*, realice la búsqueda binaria de *x*.

18. Dada una secuencia de enteros elabore un algoritmo recursivo que calcule la cantidad de números positivos y la cantidad de números negativos de la misma.

19. Diseñe un algoritmo recursivo que invierta los números de un arreglo de enteros.

20. Dada una secuencia de caracteres, elabore un algoritmo recursivo que produzca la secuencia invertida.

21. Dadas dos secuencias de caracteres, elabore un algoritmo recursivo que determine si dichas secuencias son iguales.

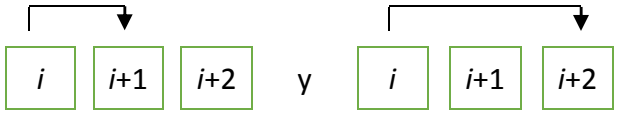
22. Dada una secuencia de caracteres, elabore un algoritmo recursivo que determine si dicha secuencia es palíndromo. *Ejemplos de palíndromos:* arepera, reconocer, oso, somos.

23. Diseñe un algoritmo recursivo que ordene de menor a mayor un arreglo de enteros basándose en la siguiente idea: Coloque el menor elemento en la primera ubicación, y luego ordene el resto del arreglo con una llamada recursiva.

24. Se tiene un camino de mosaicos. Cada mosaico está numerado, comenzando desde el 0, como se ve a continuación:



Los jugadores juegan sobre el camino desplazándose desde un mosaico al siguiente, pero pueden también saltar un mosaico. El juego está basado entonces en dos desplazamientos elementales:



Al comienzo, los jugadores se encuentran en el mosaico 0. El problema consiste el encontrar el número de caminos posibles para alcanzar un mosaico *n* cualquiera.

25. La forma para calcular cuantas maneras diferentes tengo para elegir *m* cosas distintas de un conjunto de *n* cosas es:

$$C(n,m) = \frac{n!}{m! \times (n-m)!}$$

Descubra una versión recursiva de la fórmula anterior y diseñe un algoritmo con dicha fórmula.

26. Un gusanito está en el fondo de un pozo de unos cuantos metros de profundidad (por ejemplo 6 metros) y su intención es subirlo, con la luz del sol él puede subir una cierta cantidad (por ejemplo 3 metros) pero en la noche, mientras duerme resbala una distancia determinada (por ejemplo 1 metro). En el siguiente día el gusanito no tiene la misma energía del día anterior, su condición física se ha reducido en un número específico (por ejemplo 0.3 metros), con respecto al día anterior, entonces ahora no sube 3 metros sino 2.7 metros. El deber de usted consiste en desarrollar una función recursiva que diga, si el gusanito logra salir del pozo, y en cuantos días lo hace, dependiendo de 4 valores reales los cuales son los parámetros de la función P (profundidad del pozo en metros), D (cantidad de metros que sube en el día), N (cantidad de metros que resbala en la noche) y R (cantidad de metros que deja de subir por cansancio con respecto al día anterior). Si el gusanito no puede subir el pozo debe decir en cuantos días se da por vencido.

27. Se define la Secuencia Generalizada de Fibonacci de f_0 y f_1 como la secuencia

$$\text{gfib}(f_0, f_1, 0), \text{gfib}(f_0, f_1, 1), \text{gfib}(f_0, f_1, 2), \text{gfib}(f_0, f_1, 3), \ldots, \text{gfib}(f_0, f_1, n-1), \text{gfib}(f_0, f_1, n)$$

Donde:

$\text{gfib}(f_0, f_1, 0) = f0$
$\text{gfib}(f_0, f_1, 1) = f1$
$\text{gfib}(f_0, f_1, \text{n}) = \text{gfib}(f_0, f_1, n-1) + \text{gfib}(f_0, f_1, n-2)$ si $n > 1$

Escriba una función recursiva y una iterativa para calcular $\text{gfib}(f_0, f_1, n)$.

28. Diseñe e implemente un algoritmo que imprima todas las posibles descomposiciones de un número natural como suma de números menores que él.

Ejemplos:

1 = 1
2 = 2
2 = 1+1
3 = 3
3 = 2+1
3 = 1+1+1
4 = 4
4 = 3+1
4 = 2+2
4 = 2+1+1
4 = 1+1+1+1
…
n = (n-1) +1
n = (n-2) + 2 = (n-2) + 1 + 1

29. **Las Torres de Hanoi:** El juego, en su forma más tradicional, consiste en tres varillas verticales y un conjunto indeterminado de discos de radios diferentes (tradicionalmente elaborados en madera). Al iniciar el juego todos los discos se encuentran apilados de mayor a menor radio en una de las varillas (normalmente la primera), quedando las otras dos varillas vacantes. El juego consiste en pasar todos los discos de la varilla ocupada (es decir la que posee la torre) a una de las otras varillas vacantes. Para realizar este objetivo, es necesario seguir tres simples reglas:



a) Sólo se puede mover un disco cada vez.
b) Un disco de mayor tamaño no puede descansar sobre uno más pequeño.
c) Sólo se puede desplazar el disco que se encuentre en la parte superior de cada varilla.

Realice un algoritmo recursivo que permita resolver este problema.